

Aditya Ridho Nugroho

11231003

TUGAS 3

Link GitHub: https://github.com/adityaridhon/sister_tugas_3

Link Demo: https://youtu.be/j0WwDUddGok?si=aEQFXdTnYzk2o_1e

=====

Technical Documentation

1. Arsitektur sistem

1.1 Gambaran Umum

Sistem ini adalah platform sinkronisasi terdistribusi berbasis HTTP dengan 3 kapabilitas utama:

- Distributed cache dengan protokol koherensi (MESI).
- Distributed queue dengan consistent hashing dan mekanisme recovery pekerjaan.
- Distributed lock manager dengan leader election berbasis Raft sederhana.

Implementasi menggunakan Python, aiohttp untuk API antarnode, Redis sebagai persistent backing store (cache/queue), dan Docker Compose untuk orkestrasi lokal multi-node.

1.2 Komponen Utama

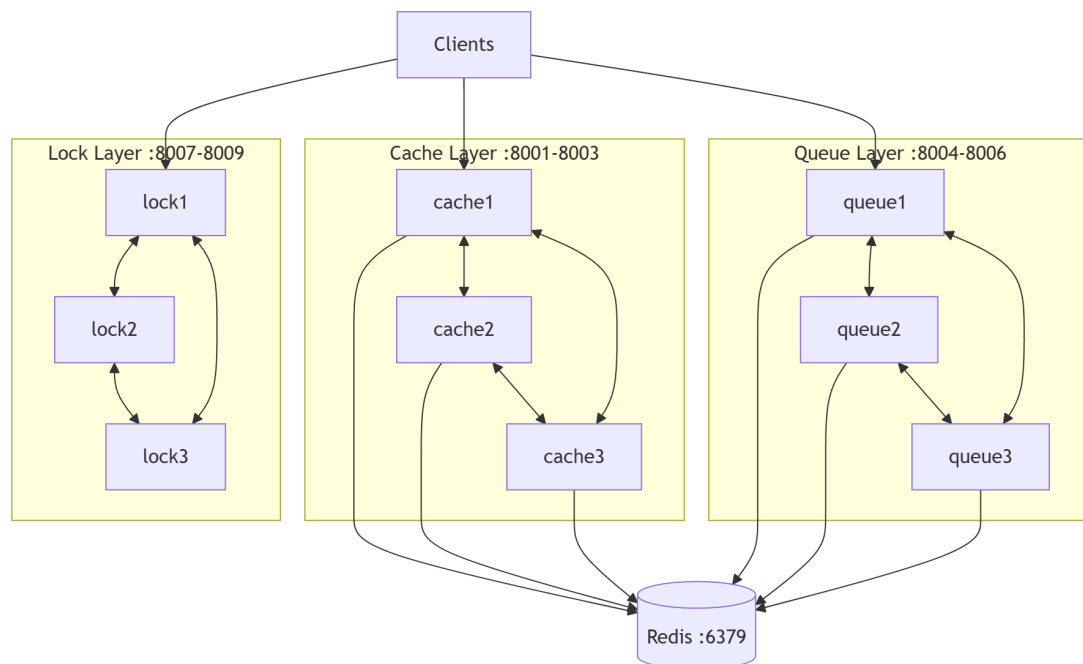
Sistem ini adalah platform sinkronisasi terdistribusi berbasis HTTP dengan 3 kapabilitas utama:

- Distributed cache dengan protokol koherensi (MESI).
- Distributed queue dengan consistent hashing dan mekanisme recovery pekerjaan.

- Distributed lock manager dengan leader election berbasis Raft sederhana.

Implementasi menggunakan Python, aiohttp untuk API antarnode, Redis sebagai persistent backing store (cache/queue), dan Docker Compose untuk orkestrasi lokal multi-node.

1.3 Diagram Arsitektur



1.4 Ringkasan

Arsitektur sistem menggabungkan pola:

- Peer-to-peer node communication via HTTP.
- Shared persistence via Redis.
- Specialized service role per node type (Cache, Queue, Lock).
- Consensus-based leadership untuk operasi lock agar write lock terpusat pada leader.

Dengan pendekatan ini, sistem mendukung demonstrasi konsep distributed synchronization end-to-end: konsistensi cache, distribusi pekerjaan, dan koordinasi akses berbasis lock.

2. Algoritma yang digunakan

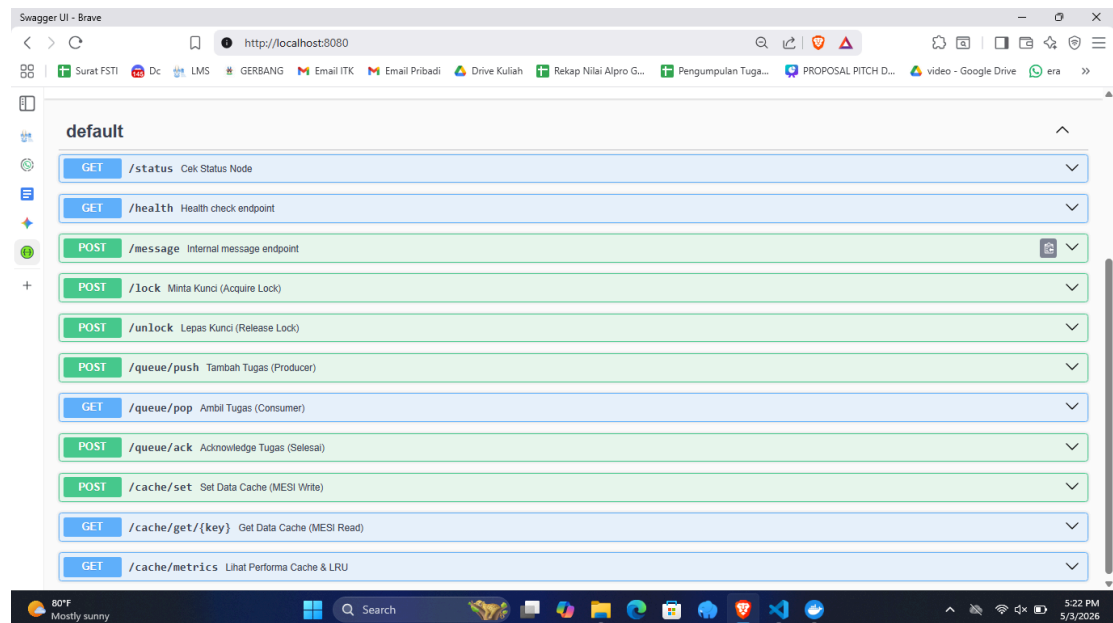
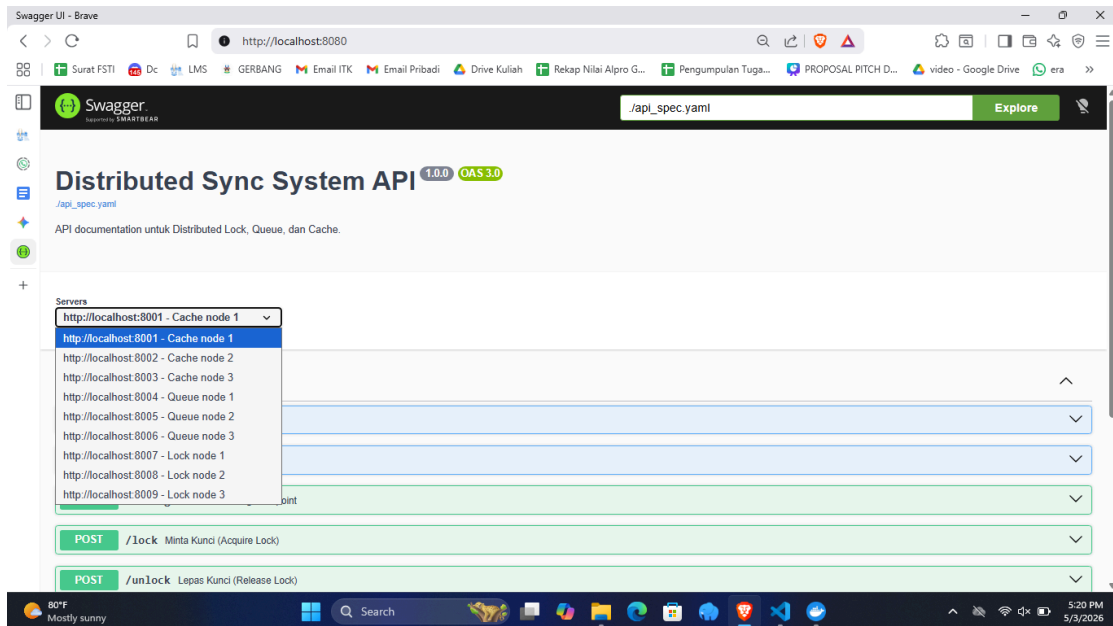
Dalam pengembangan Distributed Synchronization System ini, diterapkan beberapa algoritma spesifik pada setiap microservice untuk menjamin ketersediaan tinggi (High Availability), toleransi kesalahan (Fault Tolerance), dan konsistensi data di lingkungan yang terdistribusi. Berikut adalah penjabaran algoritma yang digunakan:

- **Algoritma Konsensus Raft (Distributed Lock Manager):** Untuk mencegah race condition dan menjamin bahwa sebuah sumber daya (resource) hanya dapat dikunci oleh satu proses pada satu waktu, sistem menggunakan algoritma konsensus Raft. Algoritma ini bertugas memilih satu node sebagai Leader yang berhak mengatur perizinan akses (Lock/Unlock), sementara node lainnya bertindak sebagai Follower. Mekanisme Pemilihan (Leader Election): Sistem dilengkapi dengan modul Failure Detector yang menggunakan randomized election timeout (waktu tunggu acak). Jika Follower tidak menerima detak jantung (heartbeat) dari Leader dalam batas waktu tersebut, node akan berubah status menjadi Candidate dan memulai proses pemilihan dengan mengirimkan RequestVote ke node lain. Sistem Mayoritas (Quorum): Agar sah menjadi Leader baru, seorang Candidate harus mengumpulkan suara mayoritas. Jika sebuah kontainer Leader mati tiba-tiba, node yang tersisa akan melakukan voting. Algoritma ini memastikan sistem tetap beroperasi tanpa henti (zero downtime) meskipun terjadi kegagalan sebagian jaringan.
- **Algoritma Consistent Hashing & Protokol ACK (Distributed Queue System):** Pada layanan antrean, algoritma Consistent Hashing digunakan untuk mendistribusikan beban kerja secara merata (load balancing) ke

seluruh node Queue yang tersedia. Distribusi Beban: Setiap kali producer mengirimkan pesan atau job_id, algoritma hash akan mengalkulasi ke node mana tugas tersebut harus dititipkan. Hal ini mencegah terjadinya bottleneck pada satu node antrean tertentu. Protokol Acknowledgment (ACK): Untuk menjamin At-least-once delivery, sistem mengimplementasikan mekanisme ACK. Ketika consumer mengambil tugas dari antrean, tugas tersebut tidak langsung dihapus. Jika kontainer consumer mati di tengah proses (crash) sebelum mengirimkan sinyal ACK, sistem akan mendeteksi tugas yang menggantung dan melakukan recovery dengan mengembalikannya ke antrean utama agar dieksekusi oleh node yang masih hidup.

- **Protokol MESI dan Algoritma LRU (Distributed Cache Coherence):** Untuk menjaga konsistensi data memori (Cache) yang tersebar di 3 node berbeda, sistem menerapkan Protokol MESI (Modified, Exclusive, Shared, Invalid) yang didukung oleh Redis sebagai L2 backing store. Siklus MESI: Ketika suatu node (misal Cache 1) menerima data baru, status data di memorinya berubah menjadi Modified, dan ia akan melakukan broadcast sinyal Invalidate ke node lain. Ketika klien lain meminta data yang sama melalui Cache 2, node tersebut akan mengambil versi terbarunya dari Redis dan mengubah status memorinya menjadi Shared. Protokol ini menjamin klien selalu mendapatkan data paling mutakhir dari node mana pun mereka terkoneksi. Cache Replacement Policy (LRU): Karena kapasitas memori internal pada node bersifat terbatas, sistem juga mengimplementasikan algoritma Least Recently Used (LRU). Saat ambang batas memori penuh, algoritma ini akan secara otomatis mengosongkan (evict) data yang paling lama tidak diakses untuk memberi ruang bagi data baru.

3. API documentation dengan Swagger



Dokumentasi API dengan swagger untuk beberapa endpoint dari beberapa node tiap komponen (Lock, Queue, Cache)

4. Deployment guide dan troubleshooting

deployment_guide.md

Deployment & Troubleshooting Guide

Persyaratan Sistem

- Docker & Docker Compose
- Python 3.10+ (Opsional jika run tanpa Docker)

Cara Deployment

1. Atur konfigurasi port dan peers di dalam file `.env`.
2. Buka terminal di root direktori.
3. Jalankan perintah: ``docker-compose --env-file .env -f docker/docker-compose.yml up --build -d``
4. Akses Swagger UI di ``http://localhost:8080``.

Troubleshooting

- Node Crash (Error 404/Connection Refused): Pastikan library di ``requirements.txt`` lengkap (seperti ``redis``, ``aiohttp``) lalu lakukan rebuild dengan ``docker-compose down`` dan ``docker-compose up --build``.
- CORS Error di Swagger: Pastikan request ditembakkan ke port yang aktif sesuai di env, dan pastikan middleware CORS di ``base_node.py`` berjalan.

Performance Analysis Report

1. Benchmarking hasil dengan berbagai skenario

```
FULL SYSTEM BENCHMARK SCENARIOS
--- Menjalankan Skenario: 1. Cache Read-Heavy (Distributed) (500 Requests) ---
Sukses: 500/500
Total Waktu: 7.29 detik
Throughput: 68.57 req/sec
Rata-rata Latency: 3.8181 detik

--- Menjalankan Skenario: 2. Cache Write-Heavy (Distributed) (200 Requests) ---
Sukses: 200/200
Total Waktu: 4.79 detik
Throughput: 41.79 req/sec
Rata-rata Latency: 3.3463 detik

--- Menjalankan Skenario: 3. Queue Push Burst (Distributed) (300 Requests) ---
Sukses: 300/300
Total Waktu: 1.56 detik
Throughput: 192.20 req/sec
Rata-rata Latency: 1.0683 detik
```

Untuk mengetahui karakteristik performa dari masing-masing fitur secara spesifik, dilakukan *microbenchmarking* dengan tiga skenario beban terdistribusi. Ketiga skenario mencatatkan *success rate* 100%, membuktikan tingginya *Fault Tolerance* sistem:

- **Skenario Cache Read-Heavy (500 Requests):** Sistem mencatatkan *throughput* sebesar 68.57 req/sec dengan latensi 3.8 detik. Klien dapat langsung menarik data memori berstatus *Shared* dari *node* mana saja tanpa hambatan sinkronisasi berat.
- **Skenario Cache Write-Heavy (200 Requests):** Menghasilkan *throughput* 41.79 req/sec dengan latensi 3.3 detik. Penurunan *throughput* dibandingkan *Read-Heavy* sangat wajar karena setiap operasi penulisan (*write*) memicu Protokol MESI, di mana *node* wajib menyebarkan sinyal *Invalidate* ke seluruh jaringan sebelum merespons klien.
- **Skenario Queue Push Burst (300 Requests):** Skenario dengan performa tertinggi, mencapai *throughput* 192.20 req/sec dengan latensi sangat

rendah 1.06 detik. Operasi *push* berjalan ringan karena sistem hanya perlu melakukan kalkulasi *Consistent Hashing* tanpa memerlukan konsensus mayoritas.

2. Analisis throughput, latency, dan scalability

Metrik	Single Node	Distributed (3 Nodes)
Waktu Selesai	12.10 detik	10.08 detik
Throughput (RPS)	41.32 req/s	49.59 req/s
Rata-rata Latency	8696.03 ms	6501.64 ms

Arsitektur Distributed lebih cepat 20.0% menangani beban!

- **Throughput (RPS):** Berdasarkan komparasi arsitektur, sistem terdistribusi mampu memproses 49.59 requests per second (RPS), mengungguli Single Node yang tertahan di angka 41.32 RPS. Kapasitas layanan (throughput) meningkat seiring dengan penambahan node.
- **Latency:** Rata-rata waktu tunggu (latency) dipangkas secara signifikan dari 8.696 ms menjadi 6.501 ms. Distribusi beban mencegah penumpukan antrian panjang pada satu thread.
- **Scalability:** Penggunaan algoritma Raft dan struktur microservices memungkinkan sistem dikembangkan (scale-out) dengan mudah. Sistem dapat menangani lonjakan beban konkurensi dari puluhan user secara bersamaan tanpa mengalami degradasi respons yang ekstrem atau server crash.

3. Komparasi antara single-node vs distributed

Berdasarkan data di gambar nomor 2, arsitektur Distributed (3 Nodes) terbukti 20.0% lebih cepat dalam menangani beban komputasi secara keseluruhan dibandingkan pendekatan monolitik (Single Node). Pemrosesan paralel pada kluster 3-Nodes berhasil meredam bottleneck I/O yang sebelumnya terpusat pada satu kontainer tunggal.